

Evaluation of LLM Theory Lab: Independent Verification and Evidence-Based Synthesis under Common Resource Budgets

Working manuscript

September 25, 2026

First draft before model experiments.

The study design, analysis plan, and budget are proposed.

Public text revision: October 3, 2026. No model-performance results are reported.

Abstract

Language models can help develop mathematical arguments, but a convincing argument may still rely on an unnoticed error. Checking the reasoning, keeping track of assumptions, and resolving disagreement are therefore central to using these models in research. We describe LLM Theory Lab, a workflow in which one agent develops an argument, another first works independently and then reviews it, and a third decides which conclusions the evidence supports. We propose an evaluation that varies this organization while keeping the model, starting material, tools, and total resource limits the same. Newly written tasks based on finite algorithm contracts allow answers and counterexamples to be checked against independent reference implementations. An optional open-ended extension considers the quality of mathematical arguments that require human review. The main comparison is against an iterative single agent; an additional generalist panel helps distinguish the effect of specialist roles from the effect of dividing work among agents. We specify measures of final-answer correctness, retention of valid counterexamples, correction of earlier errors, and resource use before running the comparison. This draft presents the workflow and study design without model-performance results. The proposed study asks whether independent checking and structured review improve the reliability of the final answer enough to justify their use within a fixed inference budget.

1 Introduction

Scientific research depends on how work is organized as well as on individual ability. A promising conjecture needs independent scrutiny, and a counterexample matters only if it reaches those preparing the final argument. In a team of language-model agents, these concerns become design choices: which agents should work independently, when should they exchange results, and how should disagreements be resolved? Adding agents creates more opportunities to reason, but does not ensure that the final account retains the strongest evidence.

OpenAI’s September 2026 report on the Navier–Stokes Millennium Prize Problem illustrates the scale at which these choices arise. The report describes a successful group of roughly 10,000 concurrent agents, with a proposed resolution followed by Lean formalization and verification

[8]. How should such a system divide computation among proposing, checking, and integrating ideas? The report describes a research achievement without a controlled comparison of alternative organizations. We study these choices in a setting where tasks can be repeated, evidence checked again, and resource allowances made explicit.

The central question is:

Under common task-level resource ceilings, does independent verification followed by a structured review of disagreements improve the reliability of an agent laboratory’s final research artifact?

An iterative single agent provides a demanding baseline: it can inspect its own argument, run computations, and revise its answer. An independent panel offers another use of the same allowance: gather several attempts and combine them. The proposed workflow must justify its division of responsibilities and timing of disclosure by producing evidence that these alternatives miss or preserving evidence more effectively through revision. Whether it does so is an empirical question.

LLM Theory Lab addresses this question through four planned stages. A Builder develops a candidate account, while a Verifier independently reconstructs the problem and searches for flaws. Once both reports are saved, the Verifier reads the Builder’s report and audits their disagreements. A Director reaches a final decision by tracing claims to evidence. A deterministic controller must enforce access rules and budgets without mathematical advice. Recording the reports will show where evidence appears and where it is lost, allowing individual reasoning and communication rules to be studied.

The primary evaluation uses finite computer-science contracts. A candidate implementation must satisfy its contract throughout a declared domain; otherwise, the system must produce an executable counterexample. This makes the final decision externally checkable, without relying on persuasive prose or another language model’s preference. A complementary set of open-ended mathematical and algorithmic questions examines the quality of arguments, the care with which their scope is stated, and the value of the final synthesis. All tasks are independently prepared from public mathematical and algorithmic concepts.

This paper specifies the protocol, develops its evaluation, and documents the local preparation implementation. The protocol defines roles, reports, and rules for sharing evidence. The evaluation connects final correctness to the production, retention, and use of intermediate evidence. The experiment plan specifies stages, resource accounting, and a monetary budget; the implementation remains local. No model-performance experiments are reported here. The planned comparison must establish whether the workflow helps and, if so, by how much.

2 Related Work

Multiple attempts and communication. Self-consistency samples several reasoning paths and aggregates their answers [12], while multiagent debate allows agents to exchange arguments and revise answers [1]. These approaches suggest evaluating additional attempts and communication separately. Our panel begins with independent attempts and follows the treatment’s disclosure schedule. The specialist-role comparison and targeted protocol changes test whether the proposed organization adds value beyond obtaining several answers.

Revision and feedback. Self-Refine uses a model to generate feedback and revise its own output [5]. Studies of intrinsic self-correction show that revision without external feedback can fail to improve reasoning and can introduce errors [2]. These findings make the baseline’s access to evidence consequential. Our single agent receives tools, a finding ledger, and instructions to test alternatives. The comparison therefore asks whether assigning review to separate roles improves on a capable agent that can already investigate and revise its work.

Verification and inference allocation. Process supervision studies feedback on intermediate mathematical steps [4], and LeanDojo provides an interactive theorem-proving environment with explicit proof checking [13]. We use executable finite contracts for the primary endpoint and assess mathematical prose separately. Work on test-time compute scaling also makes clear that the allocation of inference effort matters [11]. We will therefore report actual token and tool use alongside the common ceilings. Comparisons at a smaller budget will use fresh runs, since truncating a completed trajectory would not show how a workflow adapts to that allowance.

Agent evaluation. Cost, reproducibility, and benchmark construction are recurring concerns in agent evaluation [3]. We address them for the research workflow as a whole: all roles draw from one resource ledger, public task packets remain separate from external graders, and tasks form the units of statistical replication. This design relates choices about agent organization and evidence sharing to outcomes that can be checked independently.

3 Research Artifacts and Evaluation Target

3.1 A task with checkable evidence

A closed task is a tuple $x = (\mathcal{D}, P, C)$: a finite input domain $\mathcal{D}$, a fixed candidate program P , and a contract predicate $C(z, P(z))$. The correct verdict is

$$v^*(x) = \begin{cases} \text{accept}, & \forall z \in \mathcal{D}, C(z, P(z)) = 1, \\ \text{refute}, & \exists z \in \mathcal{D}, C(z, P(z)) = 0. \end{cases} \quad (1)$$

The contract also treats designated execution errors and exceeding the declared deterministic step bound as violations. A contract need not have a unique valid output: interval scheduling can admit several optimal subsets, and a directed graph can have several valid topological orders or cycle certificates. The checker tests the required property, without requiring one particular answer.

Each final artifact $a = (v, w, e, L)$ contains a verdict, an optional witness, an explanation, and a finding ledger. A refutation must supply a concrete input w in $\mathcal{D}$ that reproduces a violation. For acceptance, w is null and the explanation addresses the task’s stated obligations. The primary score is

$$S(x, a) = \mathbf{1}\{\text{schema valid}\} \mathbf{1}\{v = v^*(x)\} \begin{cases} 1, & v = \text{accept}, w = \text{null}, \\ \mathbf{1}\{w \in \mathcal{D}, C(w, P(w)) = 0\}, & v = \text{refute}, \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

This score measures correctness on the declared finite domain and requires executable support for a refutation. Acceptance arguments are assessed separately: a correct decision alone cannot establish that its accompanying proof is valid.

3.2 The workflow as an intervention

Let π specify role instructions, visible history, tool access, communication edges, and stopping rules. With a fixed model configuration M and task-level budget vector b , a run produces a trajectory $\tau \sim \pi(M, x; b)$ and a final artifact $a(\tau)$. For a task set $\mathcal{T}$ fixed before evaluation, our target contrast is

$$\Delta_{F,B1}(b; \mathcal{T}) = \frac{1}{|\mathcal{T}|} \sum_{x \in \mathcal{T}} [\mathbb{E}S(x, a(\tau_F)) - \mathbb{E}S(x, a(\tau_{B1}))]. \quad (3)$$

The expectation is over repeated model executions under recorded settings. We compare complete workflows, including their allocation rules. Common aggregate ceilings give each organization the same allowance without requiring it to spend resources in the same way. Actual consumption is also an outcome of the comparison.

4 The LLM Theory Lab Protocol

4.1 Construction, independent checking, and synthesis

Figure 1 shows the planned information flow. The Builder and Verifier receive identical task packets in separate contexts. The Builder develops the strongest account of the candidate that it can support, including through self-tests. The Verifier independently reconstructs the contract, searches for counterexamples, and records obligations it cannot resolve. Neither sees the other’s report until both reports have been saved.

After the independent stage, the controller must record content hashes and the disclosure event. The Verifier then audits the Builder’s report: locate the first invalid implication, replay proposed witnesses, and trace disagreements to the responsible assumptions or computations. The Director receives both saved reports and the audit, checks decisive evidence using its own allowance, and produces the final structured artifact. It decides on evidence rather than by majority vote: one valid counterexample outweighs several unsupported claims of acceptance.

The evidence ledger stores concise claims with stable identifiers, exact assumptions, supporting computations or argument steps, and unresolved conflicts. It covers one task execution, with no memory carried between tasks. The final artifact must identify the evidence supporting its verdict. Since reports can contain errors, the Director must examine each argument before adopting its conclusion. The controller is responsible for admission, scheduling, tool execution, saving final reports, and stopping without judging the mathematics.

4.2 Visibility and resource allocation

The initial 12-response allocation is 4+4+2+2 for Builder, independent Verifier, audit, and Director. Corresponding input/output pools are 32k/8k, 32k/8k, 16k/4k, and 16k/4k tokens. Unused tokens cannot be transferred between phases. This rule fixes the allocation for replication and reserves resources for synthesis even when earlier stages consume their full shares. Roles need not use every response slot. Tool calls must pass through the controller, which must charge the call and its returned output before a subsequent model response can use the result.

Both independent reports are committed before disclosure. Role workspaces contain only the public task and explicitly permitted evidence. Grader code, labels, other tasks, development notes, and unrelated research materials remain outside them. The parent process retains credentials,

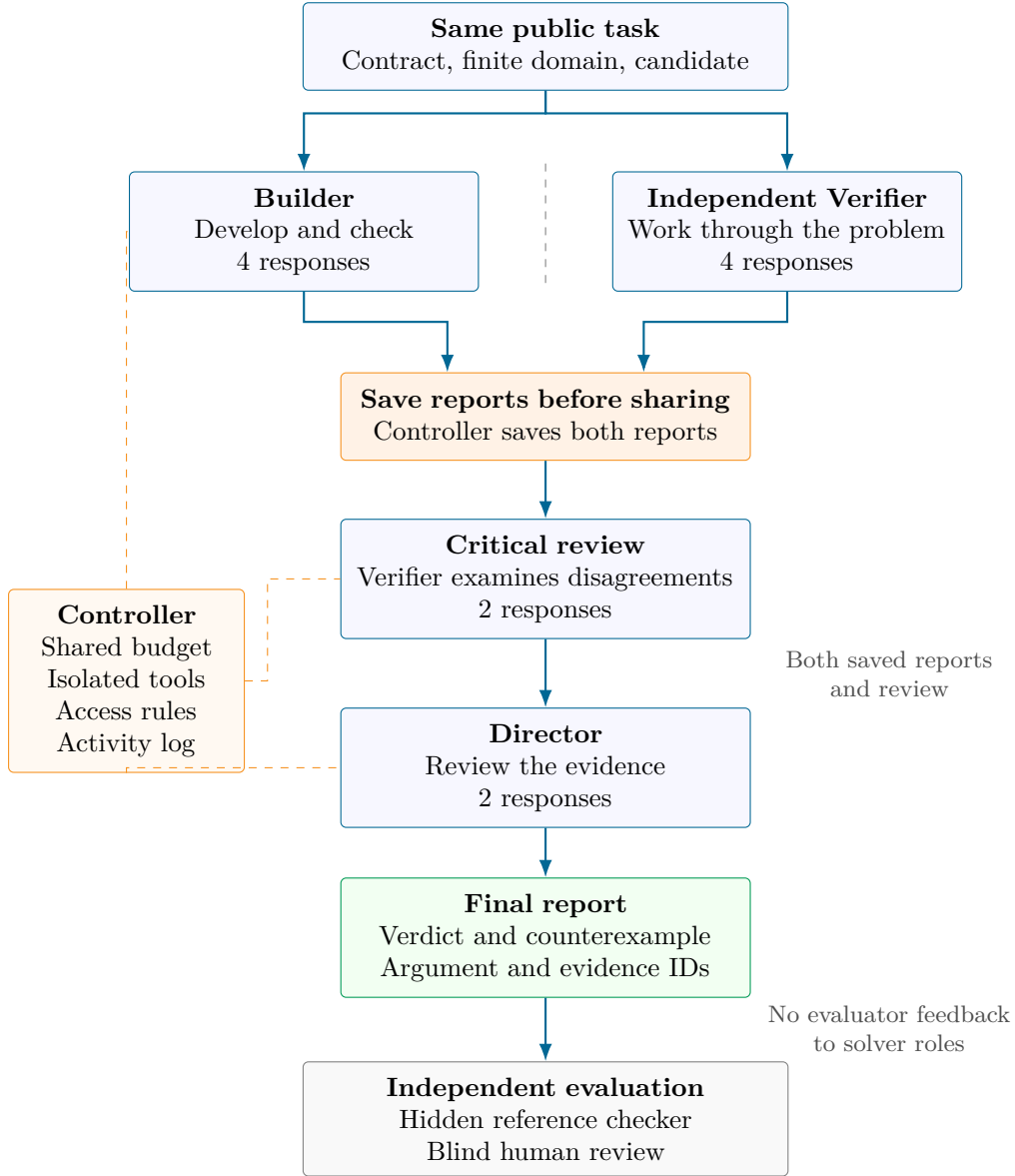


Figure 1: Planned LLM Theory Lab architecture. Solid arrows carry task evidence and reports. Dashed orange lines show the controller’s responsibilities. Builder and Verifier may run concurrently; their reports are shared only after both have been saved. The review continues in the Verifier’s context. The Director starts in a separate context with the completed reports. All four phases use the same task-level resource budget.

usage records, and the complete log. These restrictions must be enforced by the runtime; hiding information in a log viewer would not prevent a role from accessing it. The external evaluator receives the final artifact only after the run ends and returns no correctness feedback to the roles.

4.3 Mechanism: producing evidence and retaining it

Independent checking may uncover useful evidence even when the checker’s overall conclusion is wrong. Synthesis may also lose a valid witness or introduce a false claim. To distinguish improvements from such losses, consider runs with a gradeable Builder checkpoint. Let $B \in \{0, 1\}$ denote its success and $Y \in \{0, 1\}$ the Director’s success. The elementary decomposition

$$\Pr(Y = 1) - \Pr(B = 1) = \Pr(B = 0, Y = 1) - \Pr(B = 1, Y = 0) \quad (4)$$

separates corrections from cases in which an initially correct artifact becomes incorrect. If the conditioning events have positive probability, write $p = \Pr(B = 1)$, $r = \Pr(Y = 1 \mid B = 0)$, and $d = \Pr(Y = 0 \mid B = 1)$. Then

$$\Pr(Y = 1) = p + (1 - p)r - pd. \quad (5)$$

The identity tells us what to count without assuming independent errors or predicting an improvement. We will report joint counts and conditional rates, and account separately for ungradeable checkpoints. Transitions between stages within F describe what happened during a run. The F-versus-B1 comparison and the randomized workflow comparisons estimate the effect of changing the workflow.

We hypothesize that withholding the initial reports affects how much distinct evidence the two roles find, that adversarial audit affects the detection of unsupported claims, and that structured synthesis affects whether valid evidence survives. Separate contexts do not ensure independent errors when the roles use the same model. Each of these hypotheses therefore requires a direct comparison of the resulting evidence and decisions.

5 Task Suite and Independent Evaluation

5.1 Finite algorithm contracts

The development catalog contains eight pilot packets and 24 candidate main-study packets in six families (Table 1). Each packet includes a contract, a finite domain, Python source for a pure `solve(case)` function, and requirements for the accompanying argument. The catalog includes both valid and invalid candidates. Defects may involve an endpoint, an invariant, global coverage, optimality, or a certificate. Solvers receive neither truth labels nor reference implementations.

Reference implementations are written separately from the candidates and use direct checks: counting entries below a target; sorting explicit key–position pairs; enumerating feasible interval subsets; enumerating simple paths; checking orders and simple cycles; and computing connectivity in an undirected union graph. Complete enumeration determines whether each candidate satisfies its finite contract before models receive it. Certificate files record hashes of the contract, domain, candidate, and reference alongside case counts and counterexamples that can be replayed.

These tasks are intended to isolate one part of research: deciding whether a precise claim withstands scrutiny and providing evidence for that decision. They use familiar mathematical objects and algorithms drawn from public concepts. The newly written packets provide controlled

Family	Declared finite domain	Cases/packet	Pilot/main
Lower bound	Sorted arrays of length 0–8 over $\{-2, \dots, 2\}$; targets $-3-3$	9,009	2/4
Stable sorting	Keys 0, 1, 2; length 0–6; original positions retained	1,093	2/4
Interval selection	At most five distinct half-open intervals; integer endpoints 0–6	27,896	1/4
Shortest paths	Directed graphs on 1–3 vertices; absent edges or weights 0, 1, 2; every source	12,321	1/4
Order or cycle	Directed graphs on 1–4 vertices without self-loops	4,165	1/4
Union–find	Four elements; at most four union/connectivity operations	1,082,401	1/4

Table 1: Current closed-task design. Interval and graph encodings use canonical lexicographic order. Counts describe complete finite domains, not independent scientific observations. The main catalog will be finalized after the pilot and review of shared algorithm templates.

inputs, rather than new mathematical discoveries. The bounded suite is a proposed measure of claim checking. Its usefulness as a measure of research ability remains to be established; the optional open-ended study addresses that broader question.

5.2 Pilot, main freeze, and template lineage

Pilot packets are used to develop prompts, parsing, resource accounting, and grading; they are excluded from the main effect estimate. Before main-study execution, we will document the algorithm templates and defect categories used in each candidate. Some development packets currently share templates between the pilot and candidate main sets, including merge-sort tie handling, finish-time interval selection, and component-wise graph traversal. We must either replace and recertify the overlapping main templates or retain them explicitly in a preregistered study of within-family variants. The current catalog therefore cannot be described as having disjoint templates.

The core comparison concerns agent organization on the final benchmark, with equal weight for each of the six families. Testing transfer to unseen templates would require a separate split that meets that criterion. Main tasks will be selected without regard to which workflow performs best. Changes to difficulty or packet construction following the pilot will be documented before the main task manifest is finalized. A change in what the primary outcome measures requires a revised study specification. Every condition receives the same final packet for a given task.

5.3 Open-ended mathematical research layer

A separately budgeted extension uses six new research prompts, with two executions each for F and B1. Each prompt defines its objects and a central question, asks for a theorem or counterexample with explicit scope, and requests a small reproducible computational investigation where useful. Appendix C describes the intended coverage. These packets will be finalized separately from the closed study. This extension examines the coherence and usefulness of research work that cannot be assessed by a single finite acceptance decision.

Two technically qualified human reviewers independently score anonymized submissions. The rubric uses anchored 0–4 scales for correctness, rigor, completeness, reproducibility, and useful synthesis. Reviewers also record a paired preference and explain the reasons that determined it. They receive independently executed evidence and a reference dossier for each task, prepared before model outputs are read. Their initial ratings are saved before a third reviewer considers

Condition	Behavior	Response allocation
B0: one-shot	One final answer from the public packet; no adaptive tool feedback	1
B1: iterative	One continuing investigator, self-critique, local computation, finding ledger, final synthesis	Up to 12
B2: panel	Two independent generalists, neutral reconciliation, fresh generalist final synthesis	4/4/2/2
F: Theory Lab	Builder, blind Verifier, post-disclosure adversarial audit, fresh Director	4/4/2/2

Table 2: Core conditions. B1, B2, and F have the same tool opportunities and total resource limits. B0 provides a practical one-response reference; B1 is the primary control for the architecture comparison.

disagreements. False theorems count as errors even in otherwise polished submissions. LLM ratings may support an auxiliary sensitivity analysis, but do not determine mathematical truth. With six prompts, this extension provides detailed comparisons of research work, with uncertainty assessed at the prompt level.

6 Experimental Design

6.1 Questions and conditions

The primary hypothesis is that F changes complete bounded-audit success relative to a capable iterative single agent given the same aggregate resource limits. The effect defined in Equation 3 may be positive, zero, or negative. Prespecified secondary questions ask whether specialist responsibilities add value beyond an independent generalist panel and how intermediate evidence shapes the final answer.

All conditions receive the same task packet, answer schema, and general instructions for checking assumptions and providing evidence. B1 is explicitly asked to search for counterexamples and revise its reasoning. B2 follows the same independent-stage boundaries and stage sizes as F. Its generalist roles are followed by neutral reconciliation in the second generalist’s continuing context, then final synthesis by a fresh generalist. Thus, F–B2 compares the combined effects of specialist instructions, adversarial review, and evidence reconciliation; it cannot separate the effect of each instruction.

6.2 Core execution plan

The pilot consists of $8 \times 3 \times 3 = 72$ runs: eight tasks, B0/B1/F, and three repetitions. The first 24 runs cover one repetition of each task-condition pair. The remaining 48 follow once the execution checks pass. If repairs change prompts or runtime behavior, the affected pilot versions are kept separate. Progression depends on complete resource measurement, valid grading, control of report sharing, and usable output rates, rather than on a favorable ranking of conditions.

The main experiment consists of $24 \times 4 \times 3 = 288$ runs, with four tasks from each family. Before execution, we will fix the exact model identifier and version, reasoning effort, continuation policy, prompts, task manifest, reference checks, controller, sandbox image, and analysis script. The preferred option uses inference included in a Codex subscription; Section 8 describes the readiness checks and a priced API alternative. The core comparison will use one backend route throughout.

Runs are paired by task and repeat identifier. Conditions are randomized within each block and families interleaved, using ordering seed 20260926. At most two task runs may be active, with at most two concurrent model requests within each panel or Theory Lab run. Matching repeat identifiers do not imply shared provider randomness. During execution, solvers receive no cross-task transcripts, persistent solver memory, reviewer feedback, or human hints.

6.3 Resource controller and stopping rules

Full-budget runs are limited to 96,000 input tokens, 24,000 output tokens including reasoning, 12 model responses, 24 Python calls, 480 aggregate tool CPU seconds, and 1,200 elapsed seconds. B0 receives one response and no interactive Python feedback. The proposed B2/F allocations across stages are 8/8/4/4 tool calls and 160/160/80/80 CPU seconds. B1 may distribute its total allowance freely. Input accounting includes repeated conversation history, shared reports, and tool outputs. Cached input also counts toward the study’s token total.

Before each request, the controller must count the complete outgoing input and reserve the allowed output against both total and stage budgets in one atomic operation. It then reconciles this reservation with provider-reported usage. Reasoning tokens count as output and are not counted again separately. When usage is missing or uncertain, the reservation remains charged; the controller must not admit another request on the basis of unverified usage. An interface turn may contain several model responses; hidden tool cycles and retries must be accounted for before reporting a response count.

A run ends with final submission, an exhausted allowance, a timeout, or infrastructure failure. Hidden-grader success never triggers early stopping. A run that exhausts its budget is graded on its last saved final submission, or receives zero if none exists. The controller cannot make extra calls to repair an answer, and retries that generate content are disabled. A documented provider outage permits one rerun of the entire affected task-repeat block across conditions. Original attempts and their costs remain in the record. The primary intent-to-run analysis uses the original assigned attempt, including zero when no valid final submission was delivered. A delivery sensitivity analysis uses the predetermined block rerun where invoked, including rerun failures; it never selects the better attempt. The financial reserve does not authorize extra scientific samples.

6.4 Mechanism and budget interventions

Three separately priced ablations keep the 4/4/2/2 schedule: (A) show the Builder report to the Verifier before its investigation; (B) replace the adversarial audit with a constructive review of the same evidence; and (C) replace the Director’s structured decision instructions with generic synthesis. Each adds $24 \times 3 = 72$ runs, for 216 runs across all three comparisons. In A, Builder runs first and its report is available when Verifier starts. This change in concurrency must be considered when interpreting latency. Shared inputs count against the same allowance.

A half-budget experiment adds $24 \times 2 \times 3 = 144$ runs for B1/F. Every resource limit is halved, and F uses a 2/2/1/1 schedule. These are fresh executions, informed of their smaller allowance at the outset. Together with the main experiment, they provide two budget points for comparison, without a claim to a fitted scaling law. The ablations and budget comparison are preregistered as separate blocks and can be funded independently of the minimum core. Whether to run a block is decided before examining its results. Any post-main analysis is labeled exploratory, rather than retrospectively treated as confirmatory.

7 Outcomes and Analysis Plan

7.1 Final and process outcomes

The primary endpoint is Equation 2. Secondary outcomes are wrong acceptance, wrong rejection, valid-witness rate on invalid tasks, protocol failures, incomplete usage records, and time and resource consumption. Human ratings assess explanation quality separately from executable correctness. The prespecified subset is the first repeat of B1/B2/F on all 24 main tasks (run identifier 0, corresponding to $r = 1$ below): 72 submissions, each scored by two blinded reviewers, for 144 initial reviews. This secondary outcome needs a separate reviewer allocation and does not change the automated primary score. We will report task-level successes and failures for every assigned condition, including unfavorable comparisons and resource overruns.

Process outcomes examine how the workflow affects the use of evidence:

1. **Complementary evidence.** For false tasks, count valid witnesses found by either independent role and those found by exactly one. Distinguish separate findings from paraphrases using input equivalence and defect type.
2. **Evidence retention.** For runs with a valid intermediate refutation, measure whether the final submission retains a valid refutation of the same candidate. Record missing witness-bearing stages and missing final answers explicitly.
3. **Correction and loss.** Report the joint checkpoint counts in Equation 4, including cases where an initially correct answer is abandoned.
4. **Disagreement resolution.** When the saved independent reports disagree, measure final correctness and identify the evidence used to resolve the disagreement. Agreement itself is not evidence of correctness.

After a run ends, the external evaluator identifies the first valid witness; this information is never returned to the solver. Human coders classify findings without knowing the condition. Raw finding counts are descriptive, since paraphrasing a finding does not by itself add useful evidence.

7.2 Task-level estimation and uncertainty

For task i , condition c , and repeat r , let S_{icr} be the primary binary outcome. Define $\bar{S}_{ic} = \frac{1}{3} \sum_{r=1}^3 S_{icr}$ and

$$\hat{\Delta}_{F,B1} = \frac{1}{24} \sum_{i=1}^{24} (\bar{S}_{iF} - \bar{S}_{iB1}). \quad (6)$$

Because every family contains four tasks, this estimate also gives each family equal weight. We will report the contrast in absolute percentage points, scores for each condition, all 24 paired task differences, and actual resource use. The 72 executions per main-study condition represent repeated work on the same tasks, not 72 independent research problems.

The primary uncertainty summary uses 10,000 paired task-bootstrap draws, stratified within the six fixed families, with analysis seed 20260927. Each draw resamples whole task bundles containing all conditions and three repeats. Repeats are not resampled separately for this interval. This summary assumes exchangeability of tasks within each curated family; it does not describe uncertainty over all mathematical research problems. A template-cluster sensitivity analysis and

leave-one-family-out comparisons assess dependence on the chosen tasks. We will note the small number of families alongside these summaries.

F-B2 and F-B0 are prespecified secondary comparisons. Any confirmatory p -values for these comparisons use Holm adjustment within that family of tests; the main presentation emphasizes effect sizes and uncertainty. The pilot has no confirmatory significance test. A null or imprecise result does not establish equivalence. Claims about small effects would require additional independent tasks, since more repetitions of this narrow set cannot broaden its coverage. Appendix D gives a hypothetical precision calculation for planning.

7.3 Artifacts and reproducibility

For each run, we retain public inputs, prompt and code hashes, the exact model configuration, raw returned responses, usage, tool inputs and outputs, exit status, records of report sharing, saved intermediate reports with their hashes and finalization times, and the final answer. The event log has an externally retained terminal hash and event count to detect truncation. Credentials and private provider continuation state are excluded from publication. Reference evaluation and tool replay should reproduce deterministic evidence, although repeated remote inference may follow different paths. Future run records will include the completed run manifest and distinguish requested settings from those actually observed.

8 Execution Routes and Budget

8.1 Subscription allowance as an execution scenario

One possible execution route uses an included subscription allowance, with \$0 in additional spending while that allowance remains available. This is a marginal-cost scenario: it excludes the subscription fee and does not imply unlimited inference. The number of runs that fit depends on the model, task, and available quota. The September 25, 2026 design distinguishes this scenario from purchased credits and API-key billing, following the corresponding documentation [7]. Exhausting the included allowance would interrupt the schedule; it would not change the experiment’s resource limits or justify switching billing routes within a comparison.

The design considered the Codex CLI’s non-interactive interface, with JSON event streams, usage fields, and structured final outputs [9]. These features make it a candidate for automated execution, but do not establish every control the experiment needs. Exact input accounting before a request, enforceable output limits, and isolation of a single model response remain requirements to demonstrate. A usage report returned after execution cannot by itself enforce a bound beforehand. The preparation package therefore leaves live experiment dispatch disabled until these capabilities are demonstrated or the execution design is revised and documented.

Local reference enumeration, analysis calculations, software tests, and document compilation require no external model calls. They establish parts of the preparation package, rather than the performance of any workflow. Separate feasibility checks would test whether an execution route can enforce the required limits and preserve the intended information boundaries. Such checks must be identified as runtime validation and excluded from the scientific comparison. Before benchmark execution, the task manifest and execution specification must be fixed and reviewed. Development activity is not a substitute for a recorded benchmark run.

Block	Runs	Max. responses	Input M	Output M	Base USD
Pilot	72	600	6.912	1.728	31.10
Main	288	2,664	27.648	6.912	124.42
Three ablations	216	2,592	20.736	5.184	93.31
Half-budget B1/F	144	864	6.912	1.728	31.10
Open-ended extension	24	288	2.304	0.576	10.37
Pilot + main	360	3,264	34.560	8.640	155.52
All listed blocks	744	7,008	64.512	16.128	290.30

Table 3: Planned maxima and conditional GPT-6 Sol API amounts from the September 25, 2026 worksheet, with no caching discount assumed. These are run allowances, not forecasts of consumption. The separate \$0 marginal-cost scenario uses included subscription allowance and is not priced by this table.

Optional API package	Base	All-input-write scenario	Illustrative cost ceiling
Pilot alone	\$31.10	\$34.56	\$45
Pilot + main, total	\$155.52	\$172.80	\$210
All listed blocks, total	\$290.30	\$322.56	\$390

Table 4: Illustrative cost ceilings, not expenditures. Packages are cumulative alternatives: the core total already includes the pilot. Optional blocks can be considered separately. The contingency covers operational variability within the specified schedule, not additional experiments selected after seeing their outcomes.

8.2 Illustrative API cost scenario

A second scenario uses metered API calls with local Python tools. The September 25, 2026 cost worksheet uses the OpenAI Responses API with GPT-6 Sol, Standard service, and medium reasoning as an illustrative configuration. It provides a common basis for comparing the proposed study blocks; no benchmark results or actual expenditure are attributed to this configuration. An eventual experiment must record the exact returned model identifier and supported settings, preferably using a dated model version when one is available.

The worksheet assumes Standard short-context rates of \$2 per million input tokens and \$10 per million output tokens, a cache-write rate of \$2.50 per million input tokens, and a 272k input-token threshold for long-context pricing [6, 10]. These are dated planning inputs, not current price quotations. The entire per-run input allowance is 96k, below the assumed threshold if input accounting is correct. With millions of input and output tokens denoted I and O , the base planning amount is

$$C_{\text{base}} = 2I + 10O, \quad C_{\text{write}} = 2.5I + 10O. \quad (7)$$

The second expression prices every input token at the assumed cache-write rate. The illustrative cost ceilings add 20% to that scenario and round upward. They describe conditional planning bounds, rather than observed bills or expected consumption. Taxes, human annotation, and purchased infrastructure are outside these token amounts and would need separate estimates. The calculation includes no hosted tool charges.

To enforce a chosen cost ceiling, a monetary ledger must reserve the maximum charge for each complete request, including unresolved charges from earlier requests. Execution stops when another

request would exceed that ceiling. Outage reruns count toward the same total, and their costs remain visible alongside those of the original attempts. This accounting is needed even when actual use falls well below the planned maxima.

The core study consists of the pilot followed by the four-arm main comparison. Attributing an effect to a particular mechanism requires the corresponding ablation; broader claims about research artifacts require the open-ended layer and its human reviewers. Listing the blocks separately makes their scientific purposes and resource requirements clear. Before execution, the cost estimates and interface capabilities must be checked again, with any changes documented before the affected comparison begins.

8.3 Time and human review

At a 20-minute per-run ceiling, the 360-run core represents at most 120 aggregate run-hours. Two continuously available task workers would require approximately 60 scheduled hours at those ceilings, before external rate limits and outages. This is scheduling arithmetic, not a runtime prediction; many tasks may stop earlier. Pilot timings will provide a more practical estimate. B0 has no sandbox calls, leaving a core maximum of 6,336 Python calls and 35.2 tool CPU hours under the proposed limits. Human annotation time will be recorded separately in the pilot. The main explanation subset requires 144 initial reviews; this labor is unpriced and separate from API charges. For the optional open layer, 24 artifacts assessed by two reviewers require 48 initial reviews plus time to resolve disagreements. Estimating that labor cost requires reviewer availability and rates.

9 Results: Prespecified Presentation

The proposed model experiments have not been run. The tables below show how their results would be reported. Appendix E describes the completed software checks and checks of the reference answers.

9.1 Reliability under the common budget

Table 5 will report answer correctness alongside resource use. We will report the F–B1 difference and its uncertainty, the numbers of assigned and completed runs, and the task domains included in the comparison. Results for each task family and every paired comparison will accompany the aggregate estimate, whether or not they favor Theory Lab.

Condition	Success	Effect vs. B1	95% interval	Input/output	Wall time
B0	Pending	Pending	Pending	Pending	Pending
B1	Pending	Reference	—	Pending	Pending
B2	Pending	Pending	Pending	Pending	Pending
F	Pending	Pending	Pending	Pending	Pending

Table 5: Main-study reporting template. Success is the macro-average of task-level repeat means. Token distributions, calls, actual monetary costs, timeouts, and usage completeness will accompany the table.

9.2 Where intermediate evidence changes the answer

To understand the final answers, we will examine which counterexamples the agents found, which errors they corrected, and whether valid evidence survived into the final report. We will choose examples using a rule set before the runs: one correctly resolved disagreement, one lost valid counterexample if observed, and one shared error if observed, using the earliest randomized run identifier in each category. These examples will illustrate the aggregate findings rather than serve as a substitute for them.

Analysis	Required denominator	Result
Evidence retention	Runs containing an externally valid intermediate refutation	Pending
Correction/loss	All gradeable Builder checkpoints, with missing checkpoints counted separately	Pending
Disagreement resolution	Runs whose saved initial reports give different verdicts	Pending
Blindness ablation	All 24 tasks, three repeats per task, if funded	Pending
Audit and synthesis ablations	The corresponding funded task-repeat blocks	Pending
Half-budget comparison	Fresh B1/F executions at the smaller allowance, if funded	Pending

Table 6: Process and mechanism reporting template. Every conditional rate will show its numerator and denominator.

9.3 Mathematical work beyond finite verdicts

If the open-ended extension is run, we will report each prompt, reviewer scores, paired preferences, computational checks, serious mathematical errors, and decisions on reviewer disagreement. We will discuss these results separately from the finite tasks. A correct theorem under explicit conditions or a decisive counterexample can be useful progress; a broader claim without adequate support should not receive a higher score simply for sounding more complete.

10 Discussion Plan

The discussion will distinguish the overall comparison from evidence about individual design choices. A positive F–B1 difference would support the workflow on the specified tasks and resource limits. The F–B2 comparison and component studies would help assess the contribution of specialist instructions and the timing of information sharing. If the difference is small or negative, the same planned measures of error correction, evidence loss, and resource use will guide the interpretation. We will not choose a different outcome after seeing which one favors the method.

The finite tasks test whether agents can check precise claims about newly written programs. Exhaustive enumeration supplies exact answers within the stated domains, but short programs based on familiar algorithms may favor exhaustive search or recalled solution patterns. Keeping the agents’ first reports separate also does not make errors from the same model statistically independent. The optional open-ended tasks examine longer mathematical arguments, at the cost of uncertainty in human review. Neither part of this study would, by itself, establish effectiveness across mathematical research or at much larger team sizes.

We are making these plans available because the evaluation problem remains unresolved. The finite tasks offer a manageable test of error checking, but we have not found a satisfactory benchmark

for the workflow’s practical value in research. We welcome others to use the protocols, run or revise the proposed experiments, and develop better evaluations. Work that builds on these ideas or configurations can cite this manuscript and the accompanying [Theory Lab repository](#).

11 Conclusion: To Be Completed after Experiments

LLM Theory Lab separates argument construction, independent checking, critical review, and a final assessment of the evidence. This draft describes how to compare that workflow with simpler alternatives under common resource limits. The empirical conclusion remains open. After the main comparison and any additional studies are complete, it should state what changed, how large the difference was, which design choices the evidence supports, and where the findings apply.

References

- [1] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023. URL <https://arxiv.org/abs/2305.14325>.
- [2] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet, 2023. URL <https://arxiv.org/abs/2310.01798>. Version 2, revised March 14, 2024.
- [3] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter, 2024. URL <https://arxiv.org/abs/2407.01502>.
- [4] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step, 2023. URL <https://arxiv.org/abs/2305.20050>.
- [5] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhunoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback, 2023. URL <https://arxiv.org/abs/2303.17651>.
- [6] OpenAI. Openai api pricing, 2026. URL <https://developers.openai.com/api/docs/pricing>. Standard token rates, accessed September 25, 2026.
- [7] OpenAI. Pricing: Codex and chatgpt work, 2026. URL <https://learn.chatgpt.com/docs/pricing>. Official documentation, accessed September 25, 2026.
- [8] OpenAI. On the Navier–Stokes millennium prize problem. OpenAI research report, September 2026. URL <https://openai.com/index/navier-stokes-solution/>. Published September 8, 2026; updated September 10, 2026. Accessed September 25, 2026.
- [9] OpenAI. Non-interactive mode, 2026. URL <https://learn.chatgpt.com/docs/non-interactive-mode>. Codex CLI documentation, accessed September 25, 2026.
- [10] OpenAI. Gpt-6 sol model, 2026. URL <https://developers.openai.com/api/docs/models/gpt-6-sol>. Official model card, accessed September 25, 2026.

- [11] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- [12] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2022. URL <https://arxiv.org/abs/2203.11171>.
- [13] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models, 2023. URL <https://arxiv.org/abs/2306.15626>.

A Task Packet and Response Specification

A public closed packet contains a task identifier, contract text, domain dictionary, candidate source, entry point, and explicit argument obligations. Solver exports omit authoring records, split labels, intended truth labels, hidden witnesses, and grader code. The candidate program is material to investigate, not a source of instructions for the controller. Each role has access only to this packet, its own work, and reports shared under the specified protocol.

The final JSON object has exactly these fields:

```
task_id; verdict (accept or refute); witness (case object or null); explanation (string);
findings (list of id/statement/evidence objects).
```

The witness is the input case itself, not an object that wraps the case in another field. An acceptance requires null. Finding identifiers are unique within the response. Witness validation checks exact JSON types, domain membership, and execution against the independently checked contract. In particular, a Boolean is not accepted as an integer index. Explanation strings are stored for review and are never executed as grader code.

Illustrative pilot packet. One lower-bound candidate uses $hi = len(array) - 1$, halves an interval while $lo < hi$, and returns lo . Its contract asks for the first index whose value is at least the target, or n if no such index exists. An input array $[-2]$ and target -1 requires index 1. The example shows why an otherwise familiar search loop still needs a boundary argument. It belongs to the development split and is never a main-study sample or an unseen test for a solver that has read this manuscript.

B Role Instructions and Controller Algorithm

Common instruction. Investigate the supplied finite contract. State assumptions explicitly, seek valid counterexamples, distinguish finite evidence from an unbounded proof, and give evidence for substantive findings. Respect nonunique correct outputs and the declared input domain. Produce the final answer in the specified schema.

Builder. Develop an argument about whether the candidate meets its obligations. Test your own reasoning, record remaining gaps, and submit a verdict with evidence before another role’s report is available.

Blind Verifier. Reconstruct the contract independently and challenge the candidate’s assumptions and invariants. Record concrete witnesses and unresolved obligations. A familiar-looking algorithm still needs to be checked against the stated contract.

Post-disclosure audit. Continue the Verifier context after the independent report is saved and sealed. Compare both reports, identify the first faulty inference or witness, and replay the evidence that determines the outcome. State unresolved disagreements precisely.

Director. Use both saved independent reports and the audit to prepare the final response. Resolve claims through checkable evidence. Preserve the exact scope of a valid result and cite the finding or computation supporting the final verdict.

Generalist controls. B1 investigates, critiques, and revises its work in one continuing context. B2 begins with two independent generalists, continues the second generalist for neutral reconciliation after disclosure, and uses a fresh generalist to prepare the final response. B2 therefore matches F’s context continuity and schedule; the roles’ instructions are the intended difference. All model-generated repairs and summaries must fit within the assigned response and token pools.

1. Fix the task, model settings, prompts, tool environment, limits, and output schema. Create fresh role workspaces and a budget ledger owned by the parent process.
2. Admit each model request only after reserving its exact input and maximum output. Execute tools in the isolated environment; charge returned results when they enter later model inputs.
3. Run the independent initial roles. Save, seal, and hash both reports, then share only the permitted material. The external evaluator supplies no feedback at this stage.
4. Run the post-disclosure phase and final synthesis within the remaining phase allowances. Save the final JSON response and stop when a terminal rule applies.
5. Retain the sealed event count and final chain hash outside the run log. Invoke the external evaluator, save scores and resource records, and keep failed runs in the record.

C Open Research Packets and Human Review

The optional tasks ask for a precise statement, proof or refutation, boundary cases, and reproducible computations where relevant. Each packet fixes its obligations and definitions; an answer cannot substitute a different question. Table 7 summarizes the six topics. Reviewers’ reference material and scoring guidance are withheld from solvers.

ID	Research object	Required evidence
OR01	Ordered local string rewrites	Termination and a graph criterion for confluence
OR02	Fair asynchronous updates on finite lattices	Least post-solution, freshness conditions, and an event bound
OR03	Greedy change with three denominations	Canonicity criterion, least counterexample, and a proved cutoff
OR04	Linear probing with deletion	Local deletion rule, invariant, and worst-case bound
OR05	Search trees from random priorities	Exact expected depths and gap costs; discrete-tie boundary
OR06	Scheduling with release times and ties	Static optimality, dynamic ties, and conditional invariance

Table 7: Optional open-ended tasks based on public concepts. The prompts carry no claim of mathematical novelty.

Each reviewer scores five dimensions separately: correctness of claims, rigor of supporting arguments, completion of the fixed obligations, reproducibility of evidence, and usefulness of the

final synthesis. Score 0 denotes absent or invalid work; 1 major errors or unsupported essentials; 2 mixed support with material gaps; 3 a sound account with minor omissions; 4 a sound and complete account of the specified question. Task-specific anchors distinguish a finite search from a general proof, a valid counterexample from an out-of-domain one, and a narrowed theorem from a silently changed problem.

Before rating, identifiers, role names, and condition-specific wrapper text are removed using a procedure fixed in advance; mathematical contents are preserved. Reviewers record whether they can infer a condition, since writing style may reveal it. They submit individual ratings and pairwise preferences before discussing disagreements. The report will include raw ratings, agreement, final decisions, and reasons. Reviewer time is a separate resource requirement. This layer remains a proposed evaluation until qualified reviewers are assigned and their reviews are complete.

D Precision Sensitivity for Planning

A hypothetical calculation illustrates the precision available from the proposed main study. Suppose an individual paired outcome difference has standard deviation $\sigma = 0.60$, tasks are independent, and the correlation between repeats within a task is ρ . With 24 tasks and three repeats, the standard error approximation is

$$\text{SE} \approx 0.60 \sqrt{\frac{1 + 2\rho}{72}}. \quad (8)$$

A two-sided normal approximation with 5% size and 80% power gives an effect threshold near 2.80 SE. Table 8 evaluates this formula. These assumptions are illustrative and are not estimated from model results. Template dependence and small-sample non-normality can further affect precision. The calculation motivates adding independent task coverage if small effects become the target.

Assumed repeat correlation	Effective observations	Approximate effect threshold
0.00	72.0	19.8 percentage points
0.25	48.0	24.2 percentage points
0.50	36.0	28.0 percentage points
0.75	28.8	31.3 percentage points
1.00	24.0	34.3 percentage points

Table 8: Conditional planning arithmetic, not measured statistical power or a predicted advantage.

E Preparation Artifacts and Execution Readiness

At this draft, the preparation package contains newly authored candidate tasks, independently developed reference checks, atomic budget reservations, event logs, a usage parser, tool-isolation scaffolding, and an offline schedule rehearsal. The eight pilot candidates have been exhaustively checked over 1,146,987 candidate–input pairs. These computations establish the finite answer keys for the development suite. They validate part of the infrastructure and contribute no observations to the model-performance tables.

The finite-reference executor runs reviewed candidate code prepared for the benchmark. It is distinct from the tool sandbox for model-generated code. That sandbox uses an immutable Python

container image, no network, a read-only script mount, bounded scratch storage and captured output, and a wall-time limit enforced by the parent process. Readiness for model experiments additionally requires reliable aggregate CPU measurement, token admission and output limits enforced by the provider, and demonstrated separation of model contexts and filesystem access. The provider adapter contains no live dispatch path.

Artifact	State at this draft
Eight pilot task packets	Authored and independently certified over the declared finite domains
Twenty-four main candidates	Authored; template lineage review and final main selection pending
Reference graders	Implemented; executable witness replay and nonunique outputs checked
Budget and event primitives	Implemented and locally tested with race, failure, and tampering fixtures
Role scheduler	Offline fixture rehearsal; synthetic usage is explicitly labeled
Provider adapter	Usage parser and capability gate; live experiment dispatch disabled
Model performance results	No benchmark model executions in this draft
Open-ended evidence review	Packet and rubric preparation; reviewer assignments pending

Table 9: Implementation status is separate from scientific outcome evidence. Exact commands, test output, and hashes accompany the local package.

Provenance and evaluation boundary. The tasks are newly authored from public algorithmic concepts. Task and reference authors received fresh contexts and a common specification, and prepared their source files independently before integration. The specification fixes the contract and finite domain; separate implementations provide the candidate under test and the reference check. Solver packets exclude authoring notes, reference code, hidden witnesses, and development labels. This separation during preparation does not by itself establish isolation during model execution: the runtime must enforce the role boundaries described in the protocol.

Readiness before a scientific batch. Before a scientific batch, the task and prompt manifest must be fixed, labels certified, and the model configuration recorded exactly. Resource enforcement, usage accounting, and isolation of roles and tools must be demonstrated. Grading and analysis scripts, an outage rule, and an execution budget must also be specified. Table 4 gives illustrative cost ceilings for this planning step. Feasibility checks remain separate from benchmark observations. An unverified capability is a limitation to resolve before the corresponding experiment can proceed.

F Planned Figures and Final Manuscript Completion

After experiments, the manuscript will add a paired success/resource figure for F and B1, a six-family effect display, an evidence-flow transition figure, and any completed ablation and budget-point comparisons. The plots will use all assigned outcomes under the prespecified analysis. The main table and figures will be generated from the same score files. The abstract’s empirical findings and final conclusion will be written only after those results have been checked. This draft contains no illustrative model-performance numbers in place of observations.